

CH1 Fundamental 1: Number Systems and Codes



1.1 Number Systems

1.1.1 Unsigned numbers

- Definition: Numbers that are **positive only**.
- 2 representations:
 - Positional
 - **Expansion in weights**



1. Decimal numbers

- ▣ Radix-10 numbers: 0~9
- ▣ Positional: $(183)_{10}$ $(24.75)_{10}$
- ▣ Expansion in weights:
 - ▣ $183 = 1 \cdot 10^2 + 8 \cdot 10^1 + 3 \cdot 10^0$
 - ▣ $(24.75)_{10} = 2 \cdot 10^1 + 4 \cdot 10^0 + 7 \cdot 10^{-1} + 5 \cdot 10^{-2}$



2. Binary numbers

- Radix-2 numbers: 0, 1
- Positional: $(10110111)_2$ $(11000.11)_2$
- Expansion in weights:
 - $(10110111)_2 = 1 \cdot 2^7 + 1 \cdot 2^5 + 1 \cdot 2^4 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = (183)_{10}$
 - $(11000.011)_2 = 1 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^{-2} + 1 \cdot 2^{-3}$



3. Radix- R numbers

- Positional: $(R_{n-1}R_{n-2}\dots R_0.R_{-1}R_{-2}\dots R_{-m})_R$
- Expansion in weights:
 - $(R_{n-1}R_{n-2}\dots R_0.R_{-1}R_{-2}\dots R_{-m})_R$
 $= R_{n-1} * R^{n-1} + R_{n-2} * R^{n-2} + \dots R_0 * R^0 + R_{-1} * R^{-1} + R_{-2} * R^{-2} + \dots R_{-m} * R^{-m}$



4. Octal and Hexadecimal Representation

- Why using octal and hex ?
- What's the relationship between binary and octal and between binary and hex?



(1) Octal Representation

- Radix-8 numbers: 0~7
- Positional: $(267)_8$
- Expansion in weights:
 - $(267)_8 = 2 \cdot 8^2 + 6 \cdot 8^1 + 7 \cdot 8^0 = 128 + 48 + 7 = (183)_{10}$
- 1-to-1 correspondence with 3-bit binary numbers
 - 0 1 2 3 4 5 6 7
 - 000 001 010 011 100 101 110 111



(2) Hex Representation

- Radix-16 numbers: 0~9,A~F
- Positional: $(B7)_{16}$
- Expansion in weights:
 - $(B7)_{16} = 11 \cdot 16^1 + 7 \cdot 16^0 = 176 + 7 = (183)_{10}$
- Full 1-to-1 correspondence with 4-bit binary numbers

0	1	2	3	4	5	6	7
0000	0001	0010	0011	0100	0101	0110	0111
8	9	A	B	C	D	E	F
1000	1001	1010	1011	1100	1101	1110	1111



Decimal	Binary	Octal	Hexadecimal
00	00000	00	00
01	00001	01	01
02	00010	02	02
03	00011	03	03
04	00100	04	04
05	00101	05	05
06	00110	06	06
07	00111	07	07
08	01000	10	08
09	01001	11	09
10	01010	12	0A
11	01011	13	0B
12	01100	14	0C
13	01101	15	0D
14	01110	16	0E
15	01111	17	0F
16	10000	20	10
17	10001	21	11
18	10010	22	12

Figure 1.1. Numbers in different systems.

1.1.2 Conversions

- ▣ Binary Vs. Octal
- ▣ Binary Vs. Hex
- ▣ Binary Vs. Decimal
- ▣ Others



1. conversion between binary and octal

- 0 1 2 3 4 5 6 7
- 000 001 010 011 100 101 110 111
- B $\leftrightarrow$ O: Each 3-bit **group** $\leftrightarrow$ 1-digit octal
- E.g.: $(\underline{1} \ \underline{100} \ \underline{110})_2 = (146)_8$
- O $\leftrightarrow$ B: Each octal digit $\leftrightarrow$ a 3-bit binary
- E.g.: $(2751)_8 = (\underline{101} \ \underline{111} \ \underline{101} \ \underline{001})_2$



2. conversion between binary and hex

0	1	2	3	4	5	6	7
0000	0001	0010	0011	0100	0101	0110	0111

8	9	A	B	C	D	E	F
1000	1001	1010	1011	1100	1101	1110	1111

□ B → H: every **group** of 4 bits → 1 hex digit

□ H → B: every hex digit → a 4-bit binary

□ **Example:** $(\underline{110} \ \underline{0110})_2 = (66)_{16}$
 $(3A7B)_{16} = (\underline{11} \ \underline{1010} \ \underline{0111} \ \underline{1011})_2$



Questions:

- 1: Conversion between Octal and hex ?
- 2: Conversion of fraction part?



Conversion exercises:

- $(1101)_2 = (?)_8 = (?)_{16}$
- $(46)_8 = (?)_2 = (?)_{16}$
- $(AE)_{16} = (?)_2 = (?)_8$
- $(10101.11)_2 = (?)_8 = (?)_{16}$



3. B $\leftrightarrow$ D conversion

- B $\leftrightarrow$ D:
 - ◆ Expansion in weights

□ E.g.:

$$\begin{aligned}(11010.101)_2 &= (26.625)_{10} \\(11010.101)_2 \\&= 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 + 1 \times 2^{-1} \\&\quad + 0 \times 2^{-2} + 1 \times 2^{-3} \\&= 16 + 8 + 2 + 0.5 + 0.125 \\&= (26.625)_{10}\end{aligned}$$



4. D $\rightarrow$ B Conversion: integer part

- (1) Conversion of integer part:
 - ◆ Successive division by 2.
 - ◆ Record the remainder(0 or 1) after every division.
 - ◆ Repeat the front 2 steps till: quotient = 0.
 - ◆ The down-top sequence of remainders is the one!



E.g.:

$$(25)_{10} = (11001)_2$$

$2 \overline{) 25}$	余
$2 \overline{) 12}$	1
$2 \overline{) 6}$	0
$2 \overline{) 3}$	0
$2 \overline{) 1}$	1
0	1



4. D $\rightarrow$ B conversion: fraction part

- Conversion of **fraction** part:
 - ◆ Successive **multiplication** by 2
 - ◆ Record the **integer** part of product (0 or 1) after every multiplication.
 - ◆ Clear the integer part(**pure decimal**) before the next multiplication.
 - ◆ Repeat till the fraction part is **all 0s** or a **desired accuracy** is reached.
 - ◆ The **top-down** sequence of integers is the one!



Example1:

$$(0.625)_{10} = (0.101)_2$$

0.625	整
$\times 2$	
1.250	1
0.250	
$\times 2$	
0.500	0
$\times 2$	
1.000	1



fraction part is all 0s !



Example2:

Convert $(0.8254)_{10}$

$$0.8254 \times 2 = 1.6508 \quad \rightarrow \quad 1 \text{ MSB}$$

$$0.6508 \times 2 = 1.3016 \quad \rightarrow \quad 1$$

$$0.3016 \times 2 = 0.6032 \quad \rightarrow \quad 0$$

$$0.6032 \times 2 = 1.2064 \quad \rightarrow \quad 1$$

$$0.2064 \times 2 = 0.4128 \quad \rightarrow \quad 0$$

$$0.4128 \times 2 = 0.8256 \quad \rightarrow \quad 0$$

$$0.8256 \times 2 = 1.6512 \quad \rightarrow \quad 1$$

$$0.6512 \times 2 = 1.3024 \quad \rightarrow \quad 1 \text{ LSB}$$

$$(0.8254)_{10} = (0.11010011 \dots)_2$$



Question

- Conversion between decimal & octal and between the decimal & hex ?



5. Summary of conversions

- B, O, H: **Grouping**.
- Any number system $\leftrightarrow$ D: Expansion in weights.
- D $\leftrightarrow$ B and other number systems:
 - **Successive division** for the integer part.
 - **Successive multiplication** for the fraction part.

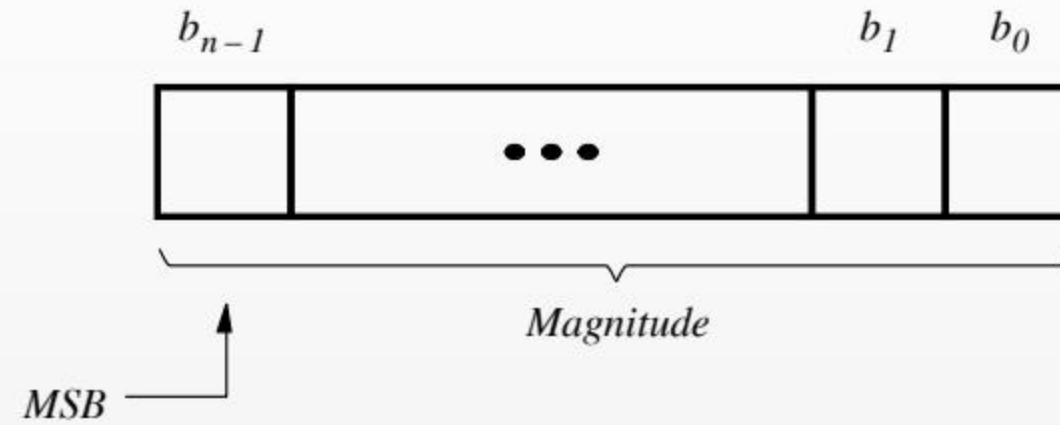


Conversion exercises:

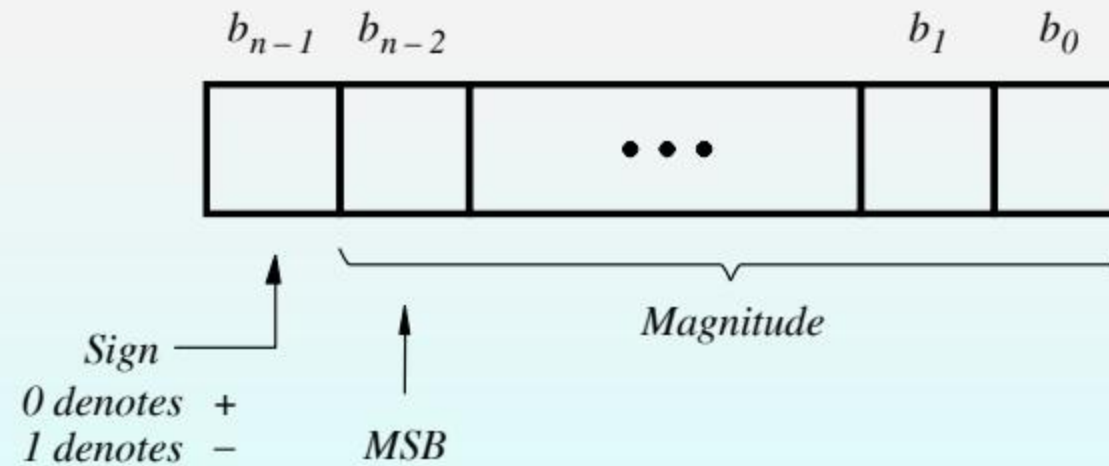
- $(11010)_2 = (?)_{10}$
- $(46)_{10} = (?)_2 = (?)_8 = (?)_{16}$
- $(0.375)_{10} = (?)_2 = (?)_8 = (?)_{16}$
- $(174.25)_{10} = (?)_2 = (?)_8 = (?)_{16}$
- $(1074.4375)_{10} = (?)_2 = (?)_8 = (?)_{16}$



1.1.3 Signed Numbers



(a) Unsigned number



Machine (Computer) number (b) Signed number

Negative numbers

- ▣ 3 representations for signed numbers:
 - ▣ Sign-and-magnitude
 - ▣ 1's complement
 - ▣ 2's complement
- ▣ The 3 patterns are the **same for positive** numbers.



Sign-and-magnitude

- *0* represents '+' for a *positive* number.
- *1* represents '-' for a *negative* number.
- Leaving all the significant bits unchanged.



1's complement for *negative numbers*

- '1' represents the sign bit
- complement all the significant bits.
- E.g.:

$$(-111)_{1's} = 1000$$



2's complement for *negative numbers*

- ▢ Suppose $K1 = 1$'s complement, $K2 = 2$'s
- ▢ $K2 = K1 + 1.$

E.g.:

$$(-111)_{2's} = 1000 + 1 = 1001$$



A quick way to find 2's complement

- After fixing the sign bit to 1...
 - For significant bits:
 - Check from right to left.
 - Copy all the '0's and the first '1'.
 - Complement the rest.



Exercises:

- $(+1101)_2 = (?)_{s\&m} = (?)_{1's} = (?)_{2's}$
- $(-10011)_2 = (?)_{s\&m} = (?)_{1's} = (?)_{2's}$
- $(110111)_{2's} = (?)_{1's} = (?)_{s\&m} = (?)_2$
- $(-0.10101)_2 = (?)_{s\&m} = (?)_{1's} = (?)_{2's}$
- $(-10000)_2 = (?)_{2's}$



Questions:

- Range of values for 1-byte unsigned numbers?
- What about signed numbers?



Table 5.1 Interpretation of four-bit signed integers.

$b_3b_2b_1b_0$	Sign and magnitude	1's complement	2's complement
0111	+7	+7	+7
0110	+6	+6	+6
0101	+5	+5	+5
0100	+4	+4	+4
0011	+3	+3	+3
0010	+2	+2	+2
0001	+1	+1	+1
<u>0000</u>	<u>+0</u>	<u>+0</u>	<u>+0</u>
<u>1000</u>	<u>-0</u>	-7	-8
1001	-1	-6	-7
1010	-2	-5	-6
1011	-3	-4	-5
1100	-4	-3	-4
1101	-5	-2	-3
1110	-6	-1	-2
1111	-7	<u>-0</u>	-1



1.1.4 Addition and Subtraction

- ▣ Sign bit must be considered!
- ▣ Efficiency is the point!



Sign-and-magnitude addition

- Consider the 2 cases:
 - Case 1: Addition of 2 numbers(operands) with opposite sign bits.
 - Case 2: Subtraction of 2 numbers with same sign bits.
- Never used!



1's complement addition

- Much better!
 - The sign bit is a part of addition.
 - Rule: $[N1]_{1's} + [N2]_{1's} = [N1+N2]_{1's}$
- 2 possibilities:
 - No carry-out occurs from the sign bit: Got it!
 - Carry-out occurs from the sign bit:
 - ◆ Add it to the sum (**wrapped around**).



$$\begin{array}{r} (+5) \\ + (+2) \\ \hline (+7) \end{array} \quad \begin{array}{r} 0101 \\ + 0010 \\ \hline 0111 \end{array}$$

$$\begin{array}{r} (-5) \\ + (+2) \\ \hline (-3) \end{array} \quad \begin{array}{r} 1010 \\ + 0010 \\ \hline 1100 \end{array}$$

$$\begin{array}{r} (+5) \\ + (-2) \\ \hline (+3) \end{array} \quad \begin{array}{r} 0101 \\ + 1101 \\ \hline 10010 \\ \text{C} \rightarrow 1 \\ \hline 0011 \end{array}$$

$$\begin{array}{r} (-5) \\ + (-2) \\ \hline (-7) \end{array} \quad \begin{array}{r} 1010 \\ + 1101 \\ \hline 10111 \\ \text{C} \rightarrow 1 \\ \hline 1000 \end{array}$$

Addition may be performed **twice** which results in **inefficiency**.



2's complement addition

- **Best choice!**
 - **Similar** to that of 1's complement. Except that,
 - ◆ If the carry-out occurs from the sign bit, just **drop it!**
 - Addition of 2 numbers is performed **only once** in any case !



$$\begin{array}{r}
 (+5) \quad 0101 \\
 + (+2) \quad +0010 \\
 \hline
 (+7) \quad 0111
 \end{array}$$

$$\begin{array}{r}
 (-5) \quad 1011 \\
 + (+2) \quad +0010 \\
 \hline
 (-3) \quad 1101
 \end{array}$$

$$\begin{array}{r}
 (+5) \quad 0101 \\
 + (-2) \quad +1110 \\
 \hline
 (+3) \quad 10011
 \end{array}$$

↑
ignore

$$\begin{array}{r}
 (-5) \quad 1011 \\
 + (-2) \quad +1110 \\
 \hline
 (-7) \quad 11001
 \end{array}$$

↑
ignore



2's complement subtraction

- Easy to convert subtraction to addition.
- $A - B = A + (-B)$
- To subtract B is to add its 2's with the sign bit being complemented.
- Example: $11 - 9 = 11 + (-9) = \dots$



$$\begin{array}{r}
 (+5) \\
 - (+2) \\
 \hline
 (+3)
 \end{array}
 \quad
 \begin{array}{r}
 0101 \\
 - 0010 \\
 \hline
 \end{array}
 \Rightarrow
 \begin{array}{r}
 0101 \\
 + 1110 \\
 \hline
 10011
 \end{array}$$

↑
ignore
e

$$\begin{array}{r}
 (-5) \\
 - (+2) \\
 \hline
 (-7)
 \end{array}
 \quad
 \begin{array}{r}
 1011 \\
 - 0010 \\
 \hline
 \end{array}
 \Rightarrow
 \begin{array}{r}
 1011 \\
 + 1110 \\
 \hline
 11001
 \end{array}$$

↑
ignore
e

$$\begin{array}{r}
 (+5) \\
 - (-2) \\
 \hline
 (+7)
 \end{array}
 \quad
 \begin{array}{r}
 0101 \\
 - 1110 \\
 \hline
 \end{array}
 \Rightarrow
 \begin{array}{r}
 0101 \\
 + 0010 \\
 \hline
 0111
 \end{array}$$

$$\begin{array}{r}
 (-5) \\
 - (-2) \\
 \hline
 (-3)
 \end{array}
 \quad
 \begin{array}{r}
 1011 \\
 - 1110 \\
 \hline
 \end{array}
 \Rightarrow
 \begin{array}{r}
 1011 \\
 + 0010 \\
 \hline
 1101
 \end{array}$$

Which one is the one?



1.2 Codes

1.2.1 Codes for signed numbers

- ▣ Sign-and-magnitude
- ▣ 1's complement
- ▣ 2's complement



1.2.2 BCD(Binary Coded Decimal)

- Designed to represent a decimal digit with a 4-bit binary group.

Decimal	BCD
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001



E.g. of conversion between decimal and BCD

- $(1991.7)_{10} = (\underline{0001} \ \underline{1001} \ \underline{1001} \ \underline{0001} . \underline{0111})_{\text{BCD}}$
- $(\underline{0110} \ \underline{1000} \ \underline{0011} . \underline{0101})_{\text{BCD}} = (683.5)_{10}$



BCD Addition

- The sum of 2 BCDs must be the BCD code.
- **Plus-6 correction** is required if sum > 9 .
 - Case 1: 4+5?
 - Case 2: 8+9?



1.2.3 Excess-3 code

- ▣ Based on BCD.

Decimal	Excess 3
0	0011
1	0100
2	0101
3	0110
4	0111
5	1000
6	1001
7	1010
8	1011
9	1100



E.g. of conversion with decimal number

$$\square (1991.7)_{10} = (\underline{0001} \ \underline{1001} \ \underline{1001} \ \underline{0001} . \underline{0111})_{\text{BCD}} = (?)_{\text{ex-3}}$$



1.2.3 Gray code

Decimal	Binary	Gray Code
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101
10	1010	1111
11	1011	1110
12	1100	1010
13	1101	1011
14	1110	1001
15	1111	1000

Adjacent codes differ in only 1 bit

Avoid **instantaneous errors**



Conversion between binary and gray code

1. B $\rightarrow$ G

- Suppose the binary is:

$$B = B_n B_{n-1} \dots B_0$$

- The converted Gray code is:

$$G = G_n G_{n-1} \dots G_0$$

- G is obtained by:

$$G_n = B_n$$

$$G_i = B_{i+1} \oplus B_i \quad (i=0, 1, \dots, n-1)$$

- Note:** ' $\oplus$ ' is the '**EXCLUSIVE OR**' logical operator (XOR)
- Rules of XOR:

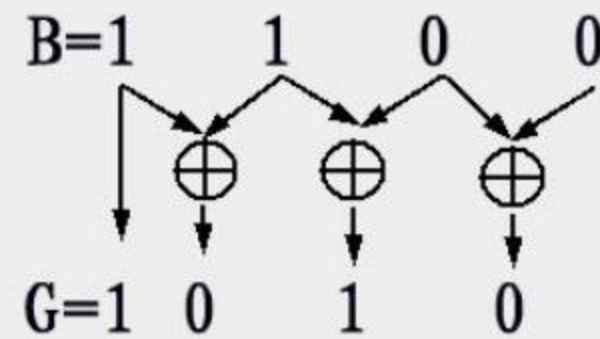
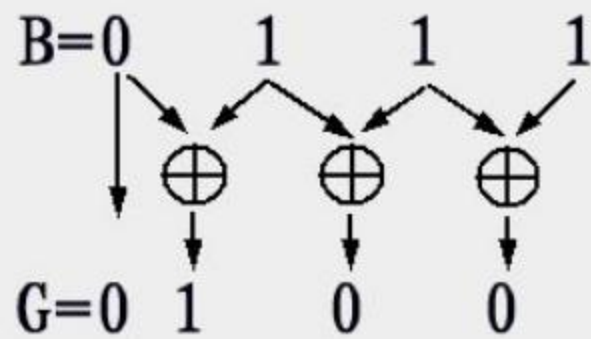
$$0 \oplus 0 = 0 \quad 0 \oplus 1 = 1 \oplus 0 = 1 \quad 1 \oplus 1 = 0$$



E.g.

$$G_n = B_n$$

$$G_i = B_{i+1} \oplus B_i \quad (i=0, 1, \dots, n-1)$$



Conversion between binary and gray code

1. $G \rightarrow B$

G is obtained by:

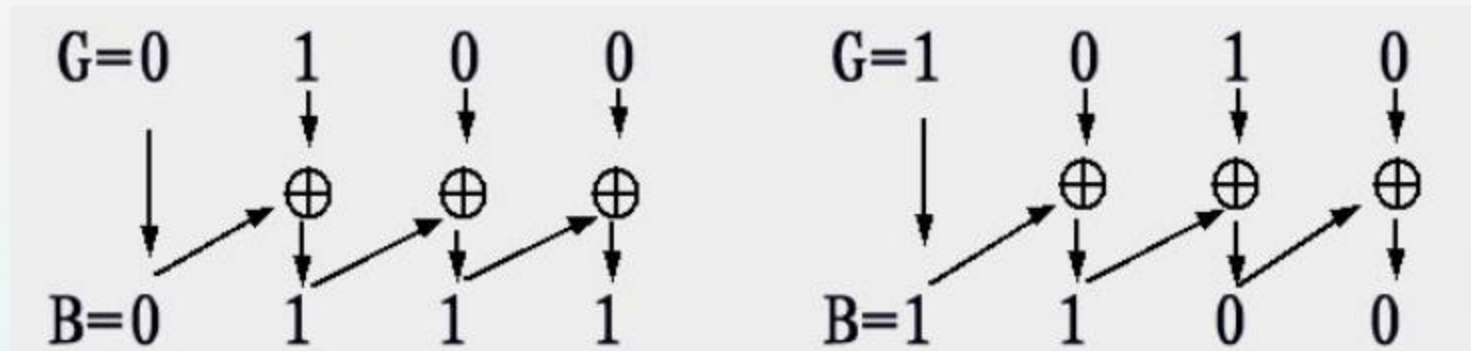
$$B_n = G_n$$

$$B_i = B_{i+1} \oplus G_i$$



E.g.

$$B_n = G_n$$
$$B_i = B_{i+1} \oplus G_i$$



1.2.4 Parity Check Code

- ▢ Code composition:
 - ▢ Data sequence + a check bit(Parity): $C_1C_2...C_nP$
- ▢ Odd or even parity:
 - ▢ Odd parity: Value of P (0 or 1) definitely makes an odd number of 1s in the code.
 - ▢ Even parity: Value of P (0 or 1) definitely makes an even number of 1s in the code.



How to get P ?

- Odd parity: $P = C_1 \oplus C_2 \oplus C_3 \oplus \dots \oplus C_n \oplus 1$
- Even parity: $P = C_1 \oplus C_2 \oplus C_3 \dots \oplus C_n$
- Why?



How to check ?

- Parity check equation:

$$S = C_1 \oplus C_2 \oplus \dots \oplus C_n \oplus P$$

- For odd parity check:
 - If $S=1$, correct!
- For even parity check:
 - If $S=0$, correct!



1.2.5 Character codes

- ASCII(American Standard Code for Information Interchange)

美国信息交换标准代码ASCII CODE								
b ₃ b ₂ b ₁ b ₀	b ₆ b ₅ b ₄							
	000	001	010	011	100	101	110	111
0000	NUL	DEL	SP	0	@	P	,	p
0001	SOH	DC1	!	1	A	Q	a	q
0010	STX	DC2	"	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EOT	DC4	\$	4	D	T	d	t
0101	ENQ	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	'	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{
1100	FF	FS	,	<	L	\	l	
1101	CR	GS	-	=	M	]	m	}
1110	SO	RS	.	>	N	↑	n	~
1111	SI	US	/	?	O	↓	o	DEL